

Scholarship Finder

Project Documentation

A Full-Stack Web Application for Scholarship Discovery and Management

Prepared by: Md Atickur Rahman

Student ID: 251035031

Course: Software Engineering

Date: 6/15/2026

Table of Contents

1.	Introduction
2.	Overall Description
3.	Functional Requirements
4.	Non-Functional Requirements
5.	Use Case Summary
6.	User Stories
7.	Data Flow Overview
8.	Assumptions and Constraints
9.	Technology Stack Matrix
10.	Conclusion

1. Introduction

Scholarship Finder is a full-stack web application designed to bridge the gap between students seeking financial support for their education and organizations offering scholarship opportunities. The platform provides a centralized space where students can discover, filter, save, and apply for scholarships, while scholarship providers can publish and manage their offerings, and administrators can oversee the integrity of the entire system.

The motivation behind this project comes from a common real-world problem: scholarship information is often scattered across multiple websites, social media pages, and institutional portals, making it difficult for students to find opportunities that match their academic background, nationality, or financial needs. Scholarship Finder addresses this problem by aggregating scholarship listings into a single, searchable, and filterable platform with role-based access for students, providers, and administrators.

This documentation describes the purpose, scope, requirements, use cases, user stories, data flow, assumptions, constraints, and technology choices for the Scholarship Finder project. It is intended to serve as a reference document for development, evaluation, and future enhancement of the system.

2. Overall Description

2.1 Product Perspective

Scholarship Finder is a standalone, web-based application built using the MERN-style architecture (MongoDB, Express.js, React.js, Node.js). It consists of a React-based frontend, a Node.js/Express backend exposing RESTful APIs, and a MongoDB database for persistent storage. File uploads such as profile pictures, transcripts, and certificates are managed through a cloud storage service (Cloudinary or similar).

2.2 Product Features (High-Level)

The system supports three primary user roles, each with a dedicated dashboard and feature set:

- Student** Browse, search, filter, save, and apply for scholarships; track application status; manage profile and documents.
- Provider** Create an organization profile, post and manage scholarship listings, review applicants, and view analytics.
- Admin** Manage users and providers, approve or reject scholarship postings, remove fake/expired listings, and view platform-wide statistics.

2.3 User Classes and Characteristics

User Class	Description	Technical Expertise
Student	Individuals seeking scholarships; uses search, filters, application tracking, and document upload.	Basic web/mobile usage
Scholarship Provider	Universities, NGOs, government bodies, or private organizations offering scholarships.	Basic to intermediate
Administrator	Platform staff responsible for moderation, user management, and analytics.	Intermediate to advanced

2.4 Operating Environment

The application is accessible through modern web browsers (Chrome, Firefox, Edge, Safari) on desktop, tablet, and mobile devices. The backend runs on a Node.js server environment and connects to a MongoDB database, which may be hosted locally during development or on a cloud provider such as MongoDB Atlas in production.

2.5 Design and Implementation Constraints

- The system must use JWT for authentication and enforce role-based access control (RBAC).
- The frontend must be built with React.js and styled using Tailwind CSS.
- All file uploads must be handled through a cloud-based storage provider.
- The system must support pagination for large data sets such as scholarship listings.

3. Functional Requirements

3.1 Authentication and Account Management

- FR-1: The system shall allow students and providers to register with role-specific information.
- FR-2: The system shall allow registered users to log in securely using email and password.
- FR-3: The system shall issue and validate JSON Web Tokens (JWT) for session management.
- FR-4: The system shall allow users to update their profile information and upload a profile picture.

3.2 Student Features

- FR-5: The system shall allow students to browse a paginated list of scholarship opportunities.
- FR-6: The system shall allow students to search scholarships by keyword.
- FR-7: The system shall allow students to filter scholarships by country, degree level, category, deadline, and funding type.
- FR-8: The system shall allow students to save scholarships to a favorites list.
- FR-9: The system shall allow students to apply for a scholarship by submitting required documents (CV, transcript, certificates).
- FR-10: The system shall allow students to view the status of their submitted applications (e.g., Pending, Under Review, Accepted, Rejected).
- FR-11: The system shall send deadline reminder notifications to students for saved or applied scholarships.
- FR-12: The system shall provide an eligibility checker that compares a student's profile against scholarship requirements.
- FR-13: The system shall allow students to view their complete application history.

3.3 Scholarship Provider Features

- FR-14: The system shall allow providers to create and manage an organization profile.
- FR-15: The system shall allow providers to create, edit, and delete scholarship postings.
- FR-16: The system shall allow providers to specify scholarship requirements, including required documents and eligibility criteria.
- FR-17: The system shall allow providers to view a list of applicants for each scholarship.
- FR-18: The system shall allow providers to accept or reject student applications.
- FR-19: The system shall provide providers with dashboard analytics showing total listings, applicants, and acceptance rates.

3.4 Administrator Features

- FR-20: The system shall provide a secure admin login separate from regular user accounts.
- FR-21: The system shall allow admins to view, activate, or deactivate student and provider accounts.
- FR-22: The system shall allow admins to approve or reject scholarship postings before they become publicly visible.
- FR-23: The system shall allow admins to remove fake, fraudulent, or expired scholarship listings.
- FR-24: The system shall provide admins with a dashboard showing platform-wide statistics and reports.

4. Non-Functional Requirements

Category	Requirement
Performance	The system shall load scholarship listing pages within 2 seconds under normal network conditions, and API responses shall complete within 500ms for typical requests.
Security	Passwords shall be hashed using a secure algorithm (e.g., bcrypt). All API endpoints requiring authentication shall validate JWT tokens, and role-based access control shall be enforced on all protected routes.
Usability	The user interface shall be intuitive, modern, and require minimal learning curve for students, providers, and admins.
Responsiveness	The application shall be fully responsive and usable on screen sizes ranging from mobile devices (320px) to desktop monitors (1920px and above).
Scalability	The backend architecture shall support horizontal scaling, and database queries shall be indexed to handle a growing number of scholarships and applications efficiently.
Reliability/Availability	The system shall target an uptime of at least 99.5%, with proper error handling to prevent crashes from invalid input or failed external service calls.
Maintainability	The codebase shall follow a clean, layered architecture (routes, controllers, services, models) with reusable components to simplify future maintenance and feature additions.
Data Validation	All user input shall be validated on both client and server sides to prevent invalid or malicious data from being stored.

5. Use Case Summary

Use Case	Actor	Description
Register Account	Student / Provider	A new user creates an account by providing personal/organizational details and selecting a role.
Login	Student / Provider / Admin	A registered user logs in with credentials and receives a JWT for authenticated access.
Search and Filter Scholarships	Student	A student searches for scholarships and applies filters such as country, degree level, category, deadline, and funding type.
Save Scholarship	Student	A student adds a scholarship to their favorites list for quick access later.
Apply for Scholarship	Student	A student submits an application along with required documents for a chosen scholarship.
Track Application Status	Student	A student views the current status of submitted applications (pending, accepted, rejected, etc.).
Check Eligibility	Student	A student checks whether their profile meets the eligibility criteria for a specific scholarship.
Post Scholarship	Provider	A provider creates a new scholarship listing with eligibility criteria, deadlines, and required documents.
Manage Scholarship Listings	Provider	A provider edits, updates, or deletes existing scholarship postings.
Review Applicants	Provider	A provider views the list of student applicants and accepts or rejects each application.
Approve/Reject Scholarship Post	Admin	An admin reviews newly submitted scholarships and approves or rejects them before publication.
Manage Users	Admin	An admin views, activates, or deactivates student and provider accounts.
View Platform Reports	Admin	An admin views dashboard statistics and analytics covering users, scholarships, and applications.

6. User Stories

6.1 Student User Stories

- As a student, I want to create an account so that I can access personalized scholarship recommendations.
- As a student, I want to search and filter scholarships by country, degree level, and funding type so that I can quickly find relevant opportunities.
- As a student, I want to save scholarships to a favorites list so that I can review them later without searching again.
- As a student, I want to upload my CV, transcript, and certificates so that I can apply for scholarships without repeatedly submitting documents.
- As a student, I want to track the status of my applications so that I know whether I have been accepted, rejected, or am still under review.
- As a student, I want to receive reminders before scholarship deadlines so that I do not miss important opportunities.
- As a student, I want to use an eligibility checker so that I only spend time applying to scholarships I qualify for.

6.2 Provider User Stories

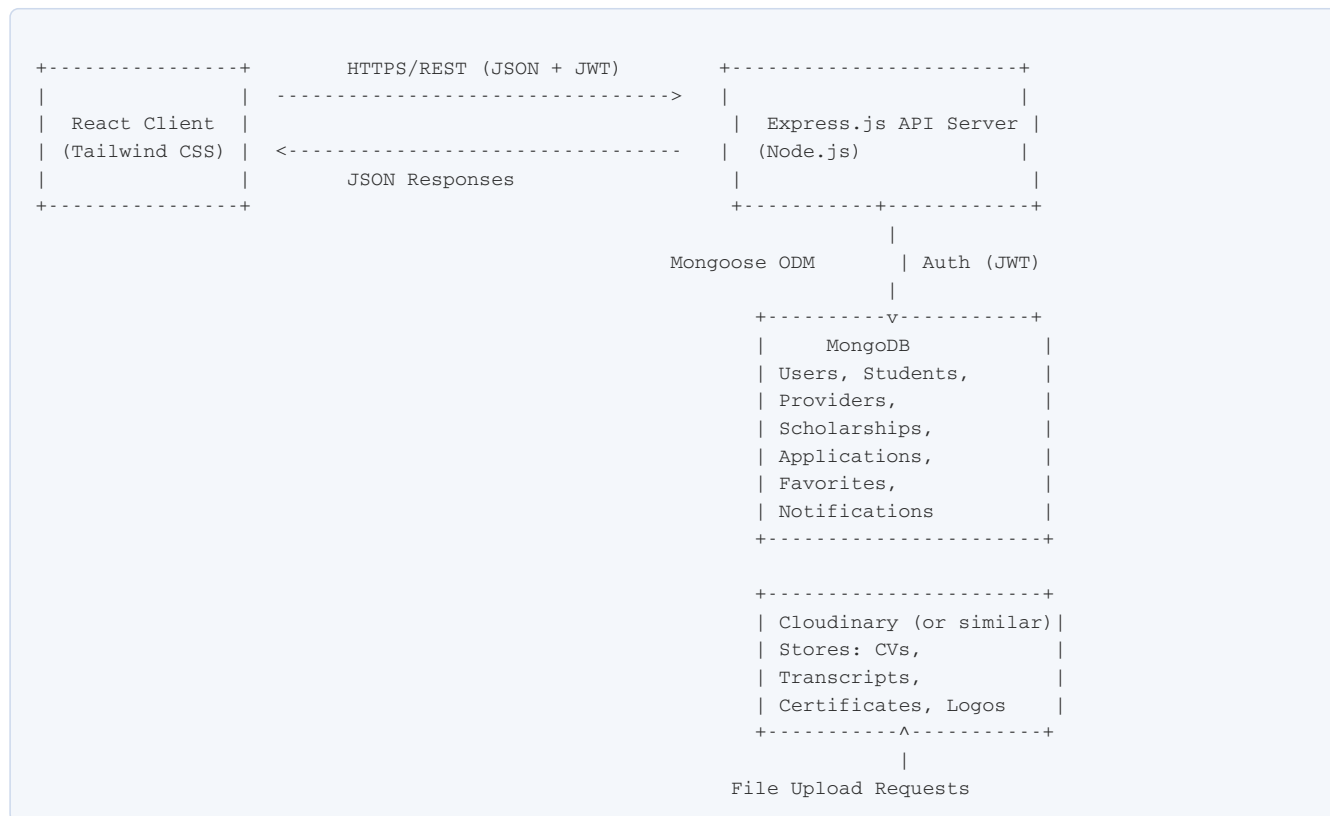
- As a provider, I want to create an organization profile so that students can recognize and trust my scholarship listings.
- As a provider, I want to post new scholarships with detailed eligibility criteria so that only suitable candidates apply.
- As a provider, I want to edit or delete my scholarship postings so that I can keep information accurate and up to date.
- As a provider, I want to view a list of applicants for each scholarship so that I can evaluate candidates efficiently.
- As a provider, I want to accept or reject applications so that I can communicate decisions to students.
- As a provider, I want to view dashboard analytics so that I can understand how my listings are performing.

6.3 Admin User Stories

- As an admin, I want to log in through a secure admin portal so that the platform's management functions are protected from unauthorized access.
- As an admin, I want to approve or reject newly posted scholarships so that only legitimate opportunities are published.
- As an admin, I want to manage student and provider accounts so that I can disable accounts that violate platform policies.
- As an admin, I want to remove fake or expired scholarships so that students only see valid, active opportunities.
- As an admin, I want to view platform-wide statistics and reports so that I can monitor growth and usage trends.

7. Data Flow Overview

The diagram below illustrates the high-level flow of data between the client, server, database, and external services used by Scholarship Finder.



7.1 Typical Application Flow (Student Applying for a Scholarship)

1. Student logs in → client receives a JWT and stores it for authenticated requests.
2. Student searches and filters scholarships → API queries the Scholarships collection in MongoDB and returns paginated results.
3. Student selects a scholarship and clicks "Apply" → client uploads required documents to Cloudinary and submits an application request to the API.
4. API creates a new record in the Applications collection, linking the Student and Scholarship documents.
5. Provider reviews the applicant list → API retrieves applications for that scholarship and returns applicant details and uploaded documents.
6. Provider updates the application status (accepted/rejected) → API updates the Applications collection and triggers a notification to the student.
7. Student views updated status on their dashboard and receives a notification.

8. Assumptions and Constraints

8.1 Assumptions

- Users have access to a stable internet connection and a modern web browser.
- Scholarship providers are assumed to provide accurate and truthful information when posting scholarships, subject to admin verification.
- Students provide accurate personal and academic information for the eligibility checker to function correctly.
- A single admin account (or small set of admin accounts) is sufficient for managing the platform during the initial release.
- Email or in-app notifications are sufficient for deadline reminders; SMS notifications are not required in the initial version.

8.2 Constraints

- The system must be developed using the specified technology stack: React.js, Tailwind CSS, Node.js, Express.js, MongoDB, and JWT-based authentication.
- File uploads (CVs, transcripts, certificates, logos) must be handled through a third-party cloud storage service due to storage limitations of the application server.
- The project timeline and resources are limited to those available for an academic Software Engineering project, which may limit the scope of advanced features in the initial release.
- All scholarship postings must pass through an admin approval process before becoming publicly visible, which may introduce a delay between posting and publication.
- The system depends on the availability and reliability of third-party services such as Cloudinary and the MongoDB hosting provider.

9. Technology Stack Matrix

Layer	Technology	Purpose
Frontend	React.js	Building reusable, component-based user interfaces for all dashboards and pages.
Frontend Styling	Tailwind CSS	Utility-first CSS framework for building a modern, responsive, and consistent UI.
Backend Runtime	Node.js	JavaScript runtime environment for executing server-side application logic.
Backend Framework	Express.js	Provides routing, middleware support, and RESTful API structure for the backend.
Database	MongoDB	NoSQL database used to store Users, Students, Providers, Scholarships, Applications, Favorites, and Notifications.
Authentication	JSON Web Tokens (JWT)	Securely authenticate users and manage role-based access control across the application.
File Storage	Cloudinary (or similar)	Stores and serves uploaded files such as CVs, transcripts, certificates, and organization logos.
Version Control	Git & GitHub	Source code management, collaboration, and branch-based development workflow.
API Communication	REST (JSON over HTTPS)	Standardized communication protocol between the React frontend and Express backend.

10. Conclusion

Scholarship Finder addresses a genuine and widespread challenge faced by students worldwide: the difficulty of discovering and applying for scholarships that match their individual profiles. By combining a searchable, filterable scholarship database with role-based dashboards for students, providers, and administrators, the platform streamlines the entire scholarship discovery and application process.

This documentation has outlined the introduction, overall description, functional and non-functional requirements, use cases, user stories, data flow, assumptions, constraints, and technology stack for the project. Together, these sections provide a clear foundation for the design, development, and evaluation of the Scholarship Finder system as part of the Software Engineering coursework.

Future development of the project may extend the platform with features such as AI-based scholarship matching, multi-language support, mobile applications, and direct messaging between students and providers, further increasing its value to both students and scholarship-providing organizations.